

Network Intrusion Detection System

Task 4: Setup, Rule Configuration, Monitoring, Response and Visualization

Item	Details
IDS Engine	Suricata (Snort 3 can be used as an alternative)
Platform	Ubuntu 22.04 / 24.04 LTS (virtual machine or physical host)
Log Format	EVE JSON (eve.json), fast.log
Response	iptables / nftables automated blocking, Suricata IPS mode
Visualization	Python + matplotlib dashboard; optional ELK or Grafana

Note: Run all attack simulations only on networks and machines you own or have written permission to test. Use an isolated lab (for example two VMs on a host-only network).

1. Objective and Overview

A Network Intrusion Detection System (NIDS) inspects network traffic and compares it with known attack signatures and behavioural patterns. When traffic matches a rule, the system generates an alert. In IPS (Intrusion Prevention System) mode it can also drop the offending packets.

This task covers five goals:

- Set up an IDS using Suricata (or Snort).
- Write rules and configure alerts for suspicious activity.
- Monitor traffic continuously.
- Implement automated response to detected intrusions.
- Visualize detected attacks on a dashboard (optional).

Snort vs Suricata

Feature	Snort 3	Suricata
Threading	Multi-threaded (Snort 3)	Multi-threaded since inception
Rule syntax	Snort rules	Snort-compatible, with extensions
Logging	Unified2, alert_json	EVE JSON (rich, structured)
Inline prevention	DAQ (afpacket, nfq)	NFQUEUE, AF_PACKET IPS
Protocol logging	Limited	HTTP, DNS, TLS, SSH, files

Suricata is used in this report because EVE JSON output makes monitoring, response automation and dashboards much simpler.

Lab Architecture

Machine	Role	Example IP
IDS Sensor	Ubuntu host running Suricata, sniffing interface eth0	192.168.56.10
Victim	Web server (Apache/Nginx) with SSH enabled	192.168.56.20
Attacker	Kali Linux with nmap, hydra, curl	192.168.56.30

For passive detection the sensor must see the traffic: use a switch SPAN/mirror port, a VirtualBox/VMware promiscuous-mode network, or install Suricata on the victim itself.

2. Installation and Setup

2.1 Install Suricata

```
sudo apt update
sudo apt install -y software-properties-common
sudo add-apt-repository ppa:oisf/suricata-stable -y
sudo apt update
sudo apt install -y suricata jq

suricata --build-info | head -n 5      # verify the installation
suricata -V
```

2.2 Identify the network interface

```
ip -br a                # e.g. eth0 or enp0s3
```

2.3 Edit /etc/suricata/suricata.yaml

Set the protected network range, the capture interface and the rule files:

```
vars:
  address-groups:
    HOME_NET: "[192.168.56.0/24]"
    EXTERNAL_NET: "!$HOME_NET"

af-packet:
  - interface: eth0
    cluster-id: 99
    cluster-type: cluster_flow
    defrag: yes

default-rule-path: /var/lib/suricata/rules
rule-files:
  - suricata.rules          # Emerging Threats Open ruleset
  - /etc/suricata/rules/local.rules  # our custom rules

outputs:
  - fast:
    enabled: yes
    filename: fast.log
  - eve-log:
    enabled: yes
    filetype: regular
    filename: eve.json
    types:
      - alert
      - http
      - dns
      - tls
      - flow
```

2.4 Download rules and start the service

```
sudo suricata-update          # fetch Emerging Threats Open rules
sudo suricata -T -c /etc/suricata/suricata.yaml -v  # test configuration
sudo systemctl enable --now suricata
sudo systemctl status suricata
```

Expected result: the test command ends with *Configuration provided was successfully loaded* and the service shows *active (running)*.

3. Rule Configuration and Alerts

3.1 Rule anatomy

```
action proto src_ip src_port -> dst_ip dst_port (options)

alert tcp any any -> $HOME_NET 22 (msg:"..."; sid:1000003; rev:1;)
```

Part	Meaning
action	alert, drop, reject, pass (drop works only in IPS mode)
header	Protocol, source/destination addresses and ports, direction
msg	Text shown in the alert
flags / flow	TCP flag match; connection state and direction
threshold	Rate-based detection (count within seconds, tracked by source)
sid / rev	Unique rule ID (use 1000000+ for local rules) and revision

3.2 Custom rules: /etc/suricata/rules/local.rules

```
# 1. ICMP ping sweep / ping detection
alert icmp any any -> $HOME_NET any (msg:"LOCAL ICMP Echo Request"; \
  itype:8; threshold:type both, track by_src, count 5, seconds 10; \
  classtype:network-scan; sid:1000001; rev:1;)

# 2. TCP SYN port scan (many SYNs from one source)
alert tcp any any -> $HOME_NET any (msg:"LOCAL Possible TCP SYN Port Scan"; \
  flags:S,12; threshold:type both, track by_src, count 20, seconds 10; \
  classtype:attempted-recon; sid:1000002; rev:1;)

# 3. SSH brute force (repeated new connections to port 22)
alert tcp any any -> $HOME_NET 22 (msg:"LOCAL SSH Brute Force Attempt"; \
  flags:S; threshold:type both, track by_src, count 5, seconds 60; \
  classtype:attempted-admin; sid:1000003; rev:1;)

# 4. SQL injection in URI
alert http any any -> $HOME_NET any (msg:"LOCAL SQL Injection Attempt"; \
  flow:established,to_server; http.uri; content:"union"; nocase; \
  content:"select"; nocase; distance:0; \
  classtype:web-application-attack; sid:1000004; rev:1;)

# 5. Cross-site scripting in URI
alert http any any -> $HOME_NET any (msg:"LOCAL XSS Attempt"; \
  flow:established,to_server; http.uri; content:"<script"; nocase; \
  classtype:web-application-attack; sid:1000005; rev:1;)

# 6. Directory traversal
alert http any any -> $HOME_NET any (msg:"LOCAL Directory Traversal"; \
  flow:established,to_server; http.uri; content:"../"; \
  classtype:web-application-attack; sid:1000006; rev:1;)

# 7. Nmap scripting engine user agent
alert http any any -> $HOME_NET any (msg:"LOCAL Nmap Scripting Engine Scan"; \
  flow:established,to_server; http.user_agent; content:"Nmap Scripting Engine"; \
  classtype:attempted-recon; sid:1000007; rev:1;)
```

3.3 Load and verify the rules

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v # must report no errors
sudo systemctl restart suricata                  # or: sudo kill -USR2 $(pidof suricata)
```

Rule summary

SID	Detects	Attack stage
1000001	ICMP echo flood / ping sweep	Reconnaissance
1000002	TCP SYN port scan	Reconnaissance
1000003	SSH brute force	Credential access
1000004	SQL injection	Web exploitation
1000005	Cross-site scripting	Web exploitation
1000006	Directory traversal	Web exploitation
1000007	Nmap NSE scanning	Reconnaissance

4. Testing the Detection (Lab Only)

Run these from the attacker VM against the victim (192.168.56.20) and watch the alerts on the sensor.

Test	Command (attacker)	Expected rule
Ping	ping -c 10 192.168.56.20	1000001
Port scan	nmap -sS -p 1-1000 192.168.56.20	1000002
SSH brute force	hydra -l testuser -P small.txt ssh://192.168.56.20 -t 4	1000003
SQL injection	curl "http://192.168.56.20/?id=1 UNION SELECT 1,2"	1000004
XSS	curl "http://192.168.56.20/?q=<script>alert(1)</script>"	1000005
Traversal	curl --path-as-is "http://192.168.56.20/../../../../etc/passwd"	1000006

Suricata also ships a quick check using a known test signature:

```
curl http://testmynids.org/uid/index.html # triggers ET 2100498 (GPL ATTACK_RESPONSE id check)
```

5. Continuous Monitoring

5.1 Live alert views

```
sudo tail -f /var/log/suricata/fast.log

# Structured alerts only, formatted:
sudo tail -f /var/log/suricata/eve.json | \
jq -c 'select(.event_type=="alert") | \
  {t:.timestamp, src:.src_ip, dst:.dest_ip, sig:.alert.signature, sev:.alert.severity}'
```

5.2 Useful summary queries

```
# Top signatures
jq -r 'select(.event_type=="alert") | .alert.signature' /var/log/suricata/eve.json \
  | sort | uniq -c | sort -rn | head

# Top attacking source IPs
jq -r 'select(.event_type=="alert") | .src_ip' /var/log/suricata/eve.json \
  | sort | uniq -c | sort -rn | head

# Engine health / packet drops
sudo suricatasc -c "dump-counters" | jq '.message.capture'
```

5.3 Keep monitoring running

- The **systemd** service restarts Suricata automatically on boot and on failure (*Restart=on-failure* is in the default unit).
- Configure **logrotate** (included in the package) so eve.json does not fill the disk.
- Re-run **suricata-update** daily with cron to refresh signatures.
- Check the **stats.log** for kernel_drops; high drops mean the sensor is overloaded.

```
# /etc/cron.daily/suricata-update
#!/bin/sh
suricata-update && kill -USR2 $(pidof suricata)
```

6. Response Mechanisms

Detection is only useful if someone acts on it. Three layers of response are implemented, from simple to strict.

Level	Mechanism	Effect
1 - Notify	Log + email/console message for every high-severity alert	Human awareness
2 - Block	Python script adds the attacker IP to an iptables/ipset blocklist	Attacker cut off from the host
3 - Prevent	Suricata IPS mode with <i>drop</i> rules (NFQUEUE)	Malicious packets never delivered

6.1 Automated blocking script (Level 1 and 2)

The script follows eve.json, counts alerts per source IP, and blocks an address once it crosses a threshold. A whitelist protects trusted hosts such as the gateway and the admin PC. Blocks expire automatically so mistakes are not permanent.

```
#!/usr/bin/env python3
# /opt/ids/auto_response.py (run as root)
import json, time, subprocess, collections, ipaddress, logging

EVE = "/var/log/suricata/eve.json"
WHITELIST = {"192.168.56.1", "192.168.56.10", "192.168.56.99"} # gateway, sensor, admin
THRESHOLD = 5 # alerts ...
WINDOW = 60 # ... within this many seconds
BLOCK_SECONDS = 900 # unblock after 15 minutes

logging.basicConfig(filename="/var/log/ids_response.log", level=logging.INFO,
                    format="%(asctime)s %(message)s")
hits = collections.defaultdict(list)
blocked = {} # ip -> unblock time

def run(cmd): subprocess.run(cmd, check=False)

def block(ip):
    run(["iptables", "-I", "INPUT", "-s", ip, "-j", "DROP"])
    blocked[ip] = time.time() + BLOCK_SECONDS
    logging.info("BLOCKED %s for %ss", ip, BLOCK_SECONDS)

def unblock_expired():
    for ip, until in list(blocked.items()):
        if time.time() >= until:
            run(["iptables", "-D", "INPUT", "-s", ip, "-j", "DROP"])
            del blocked[ip]
            logging.info("UNBLOCKED %s", ip)

def follow(path):
    with open(path) as f:
        f.seek(0, 2)
        while True:
            line = f.readline()
            if not line:
                unblock_expired()
                time.sleep(0.5)
                continue
            yield line

for line in follow(EVE):
    try:
        e = json.loads(line)
    except ValueError:
        continue
    if e.get("event_type") != "alert":
        continue
```

```

src = e["src_ip"]
try:
    ipaddress.ip_address(src)          # ignore malformed addresses
except ValueError:
    continue
if src in WHITELIST or src in blocked:
    continue
now = time.time()
hits[src] = [t for t in hits[src] if now - t < WINDOW] + [now]
logging.info("ALERT %s sev=%s %s", src, e["alert"]["severity"], e["alert"]["signature"])
if len(hits[src]) >= THRESHOLD:
    block(src)

```

6.2 Run it as a service

```

# /etc/systemd/system/ids-response.service
[Unit]
Description=IDS automatic response
After=suricata.service

[Service]
ExecStart=/usr/bin/python3 /opt/ids/auto_response.py
Restart=always

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload && sudo systemctl enable --now ids-response

```

6.3 Inline prevention with Suricata IPS (Level 3)

Run Suricata inline using NFQUEUE and change selected rules from *alert* to *drop*:

```

# Send forwarded/incoming traffic to queue 0 (keep SSH for admin access working first!)
sudo iptables -I INPUT -j NFQUEUE --queue-num 0 --queue-bypass
sudo iptables -I FORWARD -j NFQUEUE --queue-num 0 --queue-bypass
sudo suricata -c /etc/suricata/suricata.yaml -q 0

# In local.rules, convert web attack rules to prevention:
drop http any any -> $HOME_NET any (msg:"LOCAL SQL Injection BLOCKED"; \
    flow:established,to_server; http.uri; content:"union"; nocase; \
    content:"select"; nocase; distance:0; sid:1000104; rev:1;)

```

Caution: inline mode can interrupt legitimate traffic if a rule is too broad. Test rules in alert mode first, tune false positives, and only then convert to drop.

6.4 Complementary tool: Fail2ban

Fail2ban can also read the Suricata log and ban IPs, which is a ready-made alternative to the custom script. Create a filter matching the SSH brute-force signature in fast.log and a jail with *maxretry* = 3 and *bantime* = 3600.

7. Visualization Dashboard (Optional)

Two approaches are provided: a lightweight Python dashboard that works anywhere, and a production-style ELK/Grafana stack.

7.1 Python dashboard from eve.json

```
#!/usr/bin/env python3
# dashboard.py - usage: python3 dashboard.py /var/log/suricata/eve.json
import sys, json, collections
from datetime import datetime
import matplotlib.pyplot as plt

alerts = []
for line in open(sys.argv[1]):
    try:
        e = json.loads(line)
    except ValueError:
        continue
    if e.get("event_type") == "alert":
        alerts.append(e)

sigs = collections.Counter(a["alert"]["signature"] for a in alerts)
srcs = collections.Counter(a["src_ip"] for a in alerts)
hours = collections.Counter(
    datetime.strptime(a["timestamp"][:13], "%Y-%m-%dT%H").strftime("%d %H:00")
    for a in alerts)

fig, ax = plt.subplots(1, 3, figsize=(16, 4.5))
n, c = zip(*sigs.most_common(8)); ax[0].barh(n, c); ax[0].set_title("Top signatures")
n, c = zip(*srcs.most_common(8)); ax[1].bar(n, c); ax[1].set_title("Top attacker IPs")
ax[1].tick_params(axis="x", rotation=45)
k = sorted(hours); ax[2].plot(k, [hours[x] for x in k], marker="o")
ax[2].set_title("Alerts over time"); ax[2].tick_params(axis="x", rotation=45)
plt.tight_layout(); plt.savefig("ids_dashboard.png", dpi=150)
print("saved ids_dashboard.png")
```

Example output (illustrative sample data, not from a real capture):

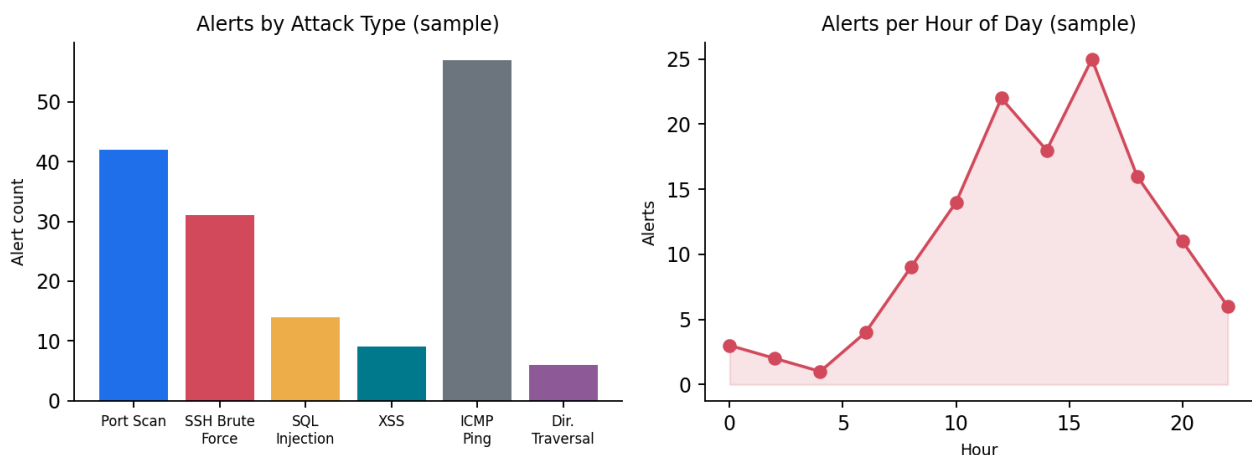


Figure 1: Sample dashboard showing alert distribution by attack type and alerts per hour.

7.2 ELK Stack / Grafana (production style)

- **Filebeat** with the Suricata module ships eve.json to Elasticsearch (*sudo filebeat modules enable suricata*).
- **Kibana** provides ready-made Suricata dashboards: alert map, top signatures, top sources, severity timeline.

- **Grafana + Loki** or Grafana + Elasticsearch is an alternative, with alert rules that notify via email, Slack or Telegram.
- Useful panels: alerts over time, top 10 attacker IPs, GeoIP world map, signature category pie chart, currently blocked IPs.

8. Validation Checklist

Requirement	How it is satisfied	Evidence to capture
Set up NIDS	Suricata installed, configured, running as a service	Screenshot of <code>systemctl status suricata</code>
Rules and alerts	7 custom rules plus Emerging Threats rules	local.rules and rule test output
Continuous monitoring	systemd service, tail -f/jq views, daily rule updates	Live fast.log or jq output during attack
Response mechanism	Auto-block script, expiring blocks, IPS drop rules	ids_response.log and <code>iptables -L INPUT -n</code>
Visualization	Python dashboard and optional Kibana/Grafana	Dashboard image

9. Troubleshooting

Problem	Likely cause and fix
No alerts appear	Wrong interface in af-packet, HOME_NET not matching, or sensor cannot see the traffic (enable promiscuous mode / SPAN port).
Rule fails to load	Run <code>suricata -T</code> ; typical issues are a missing semicolon, duplicate sid, or a buffer such as <code>http.uri</code> used with the wrong protocol.
Too many false positives	Tune thresholds, restrict rules to specific servers, or use <code>suppress</code> entries in <code>threshold.config</code> .
High packet drops	Increase threads/ring size, disable unused logging, or use a faster capture method.
Locked out after blocking	Add your admin IP to WHITELIST before starting the response script; flush with <code>iptables -F INPUT</code> from the console.

10. Conclusion

A working NIDS was designed using Suricata. It captures traffic from the protected network, matches it against community and custom rules, and records structured alerts in EVE JSON. Continuous monitoring is handled by system services and log tools, while automated blocking and optional inline prevention provide an active response to attacks. Finally, the alert data can be visualized with a Python dashboard or an ELK/Grafana stack so that analysts can quickly see attack types, sources and trends. Possible future improvements include machine-learning based anomaly detection, integration with a SIEM, and threat-intelligence feeds.

References

- Suricata documentation: docs.suricata.io
- Suricata-Update and Emerging Threats Open rules: rules.emergingthreats.net
- Snort 3 documentation: docs.snort.org
- Elastic Filebeat Suricata module documentation: elastic.co/docs